

Assignment No. : 09**Date:****Title** : Array within a Class.

Performance	Understanding	Performance Beyond defined scope	Documentation and Presentation	Timely Completion and Submission	Total	Sign & Date
5	1	1	1	1	10	

Problem Definition: Design a class Marks to store and display marks of 5 subjects using an array.

1. Add a function get Average () to calculate average marks.
2. Implement appropriate member functions.

Input: Enter the marks of 5 subjects.**Output:** Show the average marks.**Theory:****Class:**

A class is a user-defined data type in object-oriented programming (OOP) that contains data members (variables) and member functions (methods) to operate on the data. Think of a class as a blueprint for creating objects. An object is an instance of a class.

Syntax of Class (in C++):

```

Class ClassName {
    Private:
        // Data members (variables)

    Public:
        // Member functions (methods)
};

```

Example:

```
Class Student {  
    Private:  
        int rollNo  
        String name;  
  
    public:  
        void setDetails(int r, string n) {  
            rollNo = r;  
            name = n;  
        }  
  
        void displayDetails() {  
            cout << "Roll No: " << rollNo << ", Name: " << name << endl;  
        }  
};
```

Benefits of Classes:

- Encapsulation: Grouping data and functions together.
- Re-usability: Code can be reused by creating multiple objects.
- Abstraction: Hides complex implementation.
- Modularity: Easy to manage and maintain.

Arrays:

An array is a data structure used in programming to store a collection of elements of the same data type under a single variable name. It allows the programmer to store multiple values of the same type efficiently and access them using an index.

Fixed Size: The size of an array must be defined at the time of its declaration and cannot be changed during program execution. **Homogeneous Elements:** All elements in an array are of the same data type (e.g., all integers or all floats). **Indexed Access:** Elements are accessed using indices starting from 0 (zero-based indexing). **Contiguous Memory Allocation:** All array elements are stored in consecutive memory locations, which makes accessing them efficient.

Advantages of Using Arrays:

- Efficient Access: Elements can be accessed randomly using their index.
- Code Simplicity: Arrays reduce the need for multiple variable declarations.
- Structured Data Storage: Useful in storing large data sets like student marks, temperature records, etc.
- Support for Iteration: Arrays can be easily looped over using for or while loops.

Array Declaration and Initialization**Declaration:**

```
int numbers[5]; // declares an integer array of size 5
```

Initialization at declaration:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Accessing Elements:

```
cout << numbers[2]; // Output: 30 (3rd element since index starts at 0)
```

Types of Arrays

Arrays can be categorized based on their dimensions and structure. The main types are:

- One-Dimensional Array (1D)
- Two-Dimensional Array (2D)
- Multi-Dimensional Array (3D and higher)

One-Dimensional Array (1D Array)

A 1D array is the simplest type of array that stores a list of elements in a single row (or column). Each element is accessed by a single index.

Syntax:

```
data_type array_name[size];
```

Example in C++:

```
int marks[5] = {90, 85, 78, 88, 92};
```

Two-Dimensional Array (2D Array)

A 2D array is like a table or matrix with rows and columns. It requires two indices to access each element: one for the row and one for the column.

Syntax:

```
data_type array_name[rows][columns];
```

Example in C++:

```
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}};
```

Multi-Dimensional Array (3D and Higher)

A multi-dimensional array is an array with more than two dimensions. Most common is a 3D array, which can be visualized as an array of 2D arrays.

Syntax:

```
data_type array_name[size1][size2][size3];
```

Example in C++:

```
int cube[2][2][2] = {  
    {  
        {1, 2},  
        {3, 4}  
    },  
    {  
        {5, 6},  
        {7, 8}  
    }  
};
```

Common Operations on Arrays

- Traversal – Visiting each element of the array (usually using a loop).
- Insertion – Adding an element (not directly possible in fixed-size arrays without overwriting).
- Deletion – Removing an element (requires shifting other elements).
- Searching – Finding an element by value or index.
- Sorting – Arranging elements in ascending or descending order.

Limitations of Arrays

- Fixed Size: Cannot resize during runtime.
- Homogeneous Elements Only: Cannot store mixed data types.
- No Built-in Insert/Delete: Needs manual shifting of elements.

Applications of Arrays

- Storing student marks, roll numbers, or names.
- Handling matrices and mathematical computations.

- Used in algorithms like searching and sorting.
- Useful in implementing data structures like stacks and queues.

Conclusion:

This program demonstrates how to use an array within a class by creating a Marks class to store and display marks of 5 subjects. It shows the use of arrays for data storage and member functions for input, display, and calculating the average. The program effectively applies basic object-oriented programming concepts like encapsulation and modularity.

Exercise:

1. Explain multi-dimensional arrays with suitable examples.
2. Differentiate between class data members and class member functions.
3. How do arrays demonstrate the concept of structured data storage?
4. Discuss any three real-life applications of arrays.